Advanced Machine Learning

DATA 642

Lecture 13 — Regularization

*Figures and notes are from the book Mathematics for Machine Learning by A. Aldo Faisal, Cheng Soon Ong, and Marc Peter Deisenroth, Machine Learning A Bayesian and Optimization Perspective, Sergios Theodoridis, Elsevier, and Hands-On Machine Learning with Scikit-Learn Keras and TensorFlow, Aurelien Geron, O'Reilly

Avoiding Overfitting Through Regularization

Deep neural networks typically have tens of thousands of parameters, sometimes even millions. With so many parameters, the network has an incredible amount of freedom and can fit a huge variety of complex datasets. But this great flexibility also means that it is prone to **over-fitting the training set**. We need **regularization**!

ℓ_1 and ℓ_2 Regularization in Neural Networks

Regularization techniques, including ℓ_1 and ℓ_2 regularization, can be seamlessly integrated into the backpropagation algorithm, which is the standard method for training neural networks.

1. **Loss Function Modification**:
    - Regularization modifies the loss function used during backpropagation. The regularized loss function includes an additional term that penalizes large parameter values. For example, in ℓ_2 regularization, the regularized loss function is $J_{\text{reg}}(\theta) = J(\theta) + \lambda \sum_{i=1}^{n} \theta_i^2$, where $J(\theta)$ is the original loss function, and λ controls the strength of regularization.
2. **Gradient Calculation**:
    - During backpropagation, gradients are calculated not only for the original loss function but also for the regularization term. The gradients of the regularization term with respect to the parameters are added to the gradients computed from the original loss function. These additional gradients contribute to the update step of the parameters.

3. **Parameter Update**:

    - The gradients obtained from both the original loss function and the regularization term are used to update the parameters of the neural network. The update rule typically follows gradient descent principles, where the parameters are adjusted in the direction that minimizes the regularized loss function.

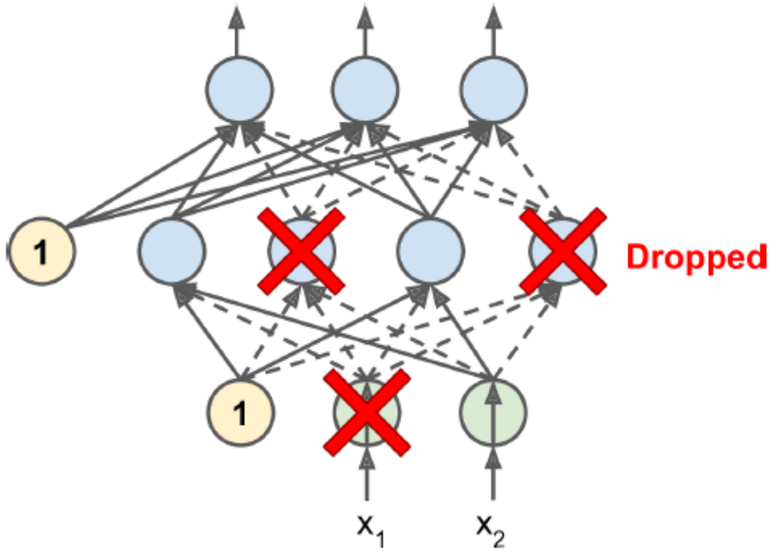
4. **Impact on Weights**:

    - Regularization affects the weight updates by shrinking the weights towards zero. This prevents the weights from growing too large during training, which helps in controlling overfitting. The regularization parameter λ controls the degree of regularization, with larger values leading to stronger regularization and more aggressive weight shrinkage.

Dropout

- Dropout is one of the most popular regularization techniques for deep neural networks. It was proposed by Geoffrey Hinton in 2012 (https://arxiv.org/abs/1207.0580).

- It is a fairly simple algorithm: at every training step, every neuron (including the input neurons, but always excluding the output neurons) has a probability p of being temporarily "dropped out," meaning it will be entirely ignored during this training step, but it may be active during the next step (see Figure 11-9). The hyper-parameter p is called the dropout rate, and it is typically set to 50%. After training, neurons do not get dropped anymore.



- If you observe that the model is over-fitting, you can increase the dropout rate.
- Conversely, you should try decreasing the dropout rate if the model under-fits the training set. It can also help to increase the dropout rate for large layers, and reduce it for small ones. Moreover, many state-of-the-art architectures only use dropout after the last hidden layer, so you may want to try this if full dropout is too strong.
- Dropout does tend to significantly slow down convergence, but it usually results in a much better model when tuned properly. So, it is generally well worth the extra time and effort.

Monte-Carlo (MC) Dropout

In 2016, a paper by Yarin Gal and Zoubin Ghahramani added more good reasons to use dropout. (https://arxiv.org/abs/1506.02142)

- First, the paper establishes a profound connection between dropout networks (i.e., neural networks containing a dropout layer before every weight layer) and approximate Bayesian inference, giving dropout a solid mathematical justification.
- Second, they introduce a powerful technique called MC Dropout, which can boost the performance of any trained dropout model, without having to retrain it or even modify it at all!
- Moreover, MC Dropout also provides a much better measure of the model's uncertainty.
- Finally, it is also amazingly simple to implement. If this all sounds like a "one weird trick" advertisement, then take a look at the following code. It is the full implementation of MC Dropout, boosting the dropout model we trained earlier, without retraining it: